分治问题

（1）逆序对问题

思想：

1. 采用分治思想，将序列 A[l..r] 分成左右两部：A[l..mid],A[mid+1..r]

2.分别递归求左半部分和右半部分的逆序对个数。

3.在合并两个有序子序列时统计跨区间逆序对。

4.若合并时有：A[i]>A[j]，由于左半部分已经有序，所以从 A[i] 到 A[mid] 的所有元素都大于 A[j]，因此产生：mid−i+1 个逆序对。

5.最终逆序对总数为：左半部分逆序对数+右半部分逆序对数+跨区间逆序对数

代码实现：

```
#include <bits/stdc++.h>
using namespace std;
long long mergeCount(vector<int>& a, vector<int>& temp, int
l, int r) {
    if (l >= r) return 0;
    int mid = (l + r) / 2;
    long long ans = 0;
    ans += mergeCount(a, temp, l, mid);
    ans += mergeCount(a, temp, mid + 1, r);
    int i = l, j = mid + 1, k = l;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) {
```

```
            temp[k++] = a[i++];
        } else {
            temp[k++] = a[j++];
            ans += mid - i + 1;
        }
    }
    while (i <= mid) temp[k++] = a[i++];
    while (j <= r) temp[k++] = a[j++];
    for (int p = l; p <= r; p++) a[p] = temp[p];
    return ans;
}

int main() {
    int n;
    cin >> n;
    vector<int> a(n), temp(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    cout << mergeCount(a, temp, 0, n - 1) << endl;
    return 0;
}
```

时间复杂度分析：

设算法时间复杂度为 T(n)。

$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = O(nlogn)$$

空间复杂度为：

$$O(n)$$

（2）最大连续子序列和问题

1. 采用分治思想，将序列 A[l..r] 分成左右两部：A[l..mid],A[mid+1..r]

2.递归求左半部分最大连续子序列和。

3.递归求右半部分最大连续子序列和。

4.从中点向左扫描求最大后缀和，从中点右侧向右扫描求最大前缀和，两者相加得到跨越中点的最大连续子序列和。

5.在三种结果中选择和最大的区段；若最大和相同，则选择长度较短的区段。

时间复杂度分析：

设算法时间复杂度为 T(n)。

$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = O(nlogn)$$

递归深度为 O(logn)，因此空间复杂度为：

$$O(logn)$$